GSD & Superpowers: Complete Reference Guide

> **Purpose:** Comprehensive reference for both AI-assisted development frameworks used in this project — what they are, how they work individually, and how to combine them for maximum effectiveness.

Table of Contents

1. Overview: Two Complementary Frameworks
2. GSD Framework
   - What Is GSD?
   - Core Concepts
   - File Artifacts & Structure
   - All Commands (Slash Commands)
   - GSD Workflow
   - Internal Subagents
3. Superpowers Framework
   - What Is Superpowers?
   - Core Concepts
   - All Skills
   - Superpowers Workflow
4. Combining Both Frameworks
   - Artifact Sharing Map
   - Decision Matrix: Which to Use When
   - Master Combined Workflow
   - Context Continuity Protocol
5. Quick Reference
6. Appendix: Optimizing CLAUDE.md for Both Frameworks
   - Why CLAUDE.md Is the Central Bridge
   - Analysis of Current CLAUDE.md
   - Recommended Enhancements
   - Enhanced Sections (Drop-in Additions)

---

1. Overview: Two Complementary Frameworks

Dimension	GSD	Superpowers
Scope	Project lifecycle management (milestones → phases → plans)	Per-session coding discipline
Persistence	Across sessions via `.planning/` files	Within session only (reads CLAUDE.md)
Invocation	Slash commands: `/gsd:progress`, `/gsd:execute-phase`	Skill tool: `superpowers:brainstorming`, `superpowers:tdd`

Dimension	GSD	Superpowers
Purpose	Strategic — what to build, when, in what order	Tactical — how to build it correctly
Agents	Spawns specialized subagents (planner, executor, verifier)	Provides discipline protocols for Claude itself
Output	Planning files, roadmaps, phase plans, summaries	Design docs, implementation plans, test suites
Memory	`.planning/STATE.md`, `ROADMAP.md`, `SUMMARY.md` files	CLAUDE.md, conversation context

The key insight: GSD handles *project management at scale* across many sessions. Superpowers handles *quality enforcement within each session*. They are designed to complement each other — GSD's executor agents receive Superpowers discipline via `.agents/skills/` and `CLAUDE.md`.

---

2. GSD Framework

2.1 What Is GSD?

GSD ("Get Shit Done") is a project lifecycle management framework for Claude Code. It provides:

- **Persistent state** across context resets via markdown files in `.planning/`
- **Structured planning** that breaks projects into milestones → phases → plans
- **Wave-based parallel execution** using specialized subagents
- **Verification loops** that check goal achievement (not just task completion)
- **Automatic routing** to the correct next action at all times

GSD is installed at `~/.claude/get-shit-done/` and exposed via slash commands.

Philosophy: Context resets are unavoidable. Every GSD command starts by loading state from files, so no prior conversation context is ever required.

2.2 Core Concepts

Hierarchy

```
Project
└── Milestone (e.g., v1.0, v1.1)
    └── Phase (e.g., Phase 4: Canvas & Workflows)
        └── Plan (e.g., 04-01: Workflow CRUD API)
            └── Task (e.g., Task 1: Write failing tests)
```

Wave-Based Execution

Plans within a phase are grouped into **waves** based on their `depends_on` frontmatter. Plans in the same wave execute in parallel. Wave 2 only starts after all Wave 1 plans complete.

```
Wave 1: Plan 01 ──┬── Plan 02 (parallel)
                  │
Wave 2:           └── Plan 03 (waits for Wave 1)
```

Autonomous vs. Checkpoint Plans

- `autonomous: true` — executor runs to completion without interruption
- `autonomous: false` — executor pauses at a checkpoint, presents to user for approval, then continues

Goal-Backward Verification

GSD verifiers check `must_haves` derived from the **phase goal**, not just whether tasks ran. A phase is complete only when the goal is demonstrably achieved in the codebase.

2.3 File Artifacts & Structure

```
.planning/
├── PROJECT.md           ← Project identity, core value, constraints, key
decisions
├── ROADMAP.md           ← Active milestone roadmap with all phases and
checkboxes
├── MILESTONES.md        ← History of all milestones and their archive
paths
├── STATE.md             ← Live project state: position, decisions,
blockers, todos
├── config.json          ← GSD settings (model profiles, parallelization,
etc.)
│
├── phases/
│   └── 04-canvas-and-workflows/
│       ├── 04-CONTEXT.md       ← User design decisions from /gsd:discuss-
phase
│       ├── 04-RESEARCH.md      ← Technical research output
│       ├── 04-VALIDATION.md    ← Nyquist validation strategy (if enabled)
│       ├── 04-01-PLAN.md       ← Executable plan with tasks in XML format
│       ├── 04-01-SUMMARY.md    ← Post-execution summary with key-files and
decisions
│       ├── 04-VERIFICATION.md  ← Goal-backward verification report
│       └── 04-UAT.md           ← User acceptance test results
│
├── milestones/
│   └── v1.0-phases/            ← Archived milestone with all its phases
│
├── quick/
│   └── N-task-slug/
```

```
  │       ├── N-PLAN.md           ← Quick task plan
  │       └── N-SUMMARY.md        ← Quick task result
  │
  ├── debug/
  │   ├── active-bug.md           ← Active debug sessions
  │   └── resolved/               ← Resolved debug sessions
  │
  └── research/                   ← Project-level research (SUMMARY.md,
STACK.md, etc.)
```

Key File Details

File	Owner	Read by	Contains
`PROJECT.md`	GSD	All agents	What, why, constraints, key decisions
`ROADMAP.md`	GSD	Planner, verifier, progress	Phase list, goals, completion status
`STATE.md`	GSD executors	All agents	Current position, decisions log, blockers, todos
`XX-PLAN.md`	`gsd-planner`	`gsd-executor`	Frontmatter + XML tasks + must_haves
`XX-SUMMARY.md`	`gsd-executor`	Orchestrator, verifier	What was built, files created, decisions made
`XX-VERIFICATION.md`	`gsd-verifier`	Orchestrator, user	Goal achievement status, gaps, human items
`XX-CONTEXT.md`	User (via discuss-phase)	Planner, checker	Design preferences, approach decisions
`XX-RESEARCH.md`	`gsd-phase-researcher`	Planner	Technical research, API choices, patterns

2.4 All Commands (Slash Commands)

`/gsd:progress`

Purpose: Check where you are and get routed to the next action.

When to use: Start of every session, or whenever you're unsure what to do next.

What it does:

1. Loads project state from `.planning/STATE.md` and `ROADMAP.md`
2. Displays: recent work, current position, key decisions, blockers, pending todos
3. Intelligently routes to the correct next command:

- Has unexecuted plans → `/gsd:execute-phase N`
- Phase needs planning → `/gsd:discuss-phase N` or `/gsd:plan-phase N`
- Phase complete, more phases remain → `/gsd:discuss-phase N+1`
- All phases complete → `/gsd:complete-milestone`
- UAT gaps found → `/gsd:plan-phase N --gaps`

Output: Rich status report + one clear next-action command.

No parameters needed.

`/gsd:new-project`

Purpose: Initialize a brand-new project from idea to ready-for-planning.

When to use: Starting a completely new project in an empty or new directory.

What it does:

1. Detects if brownfield (existing code) — optionally maps codebase first
2. Deep questioning session: purpose, users, constraints, success criteria
3. Optionally runs parallel research agents (4 concurrent) on domain/tech/architecture/pitfalls
4. Synthesizes research into `SUMMARY.md`
5. Creates `REQUIREMENTS.md` with validation
6. Creates `ROADMAP.md` with phase breakdown
7. Creates `PROJECT.md` as the project's single source of truth

Parameters:

- `--auto @doc.md` — skip questioning, synthesize from provided document

Deliverables:

- `.planning/PROJECT.md`
- `.planning/ROADMAP.md`
- `.planning/requirements/REQUIREMENTS.md`
- `.planning/research/SUMMARY.md`

`/gsd:new-milestone`

Purpose: Start a new milestone cycle after the previous one is complete.

When to use: After `/gsd:complete-milestone` archives the old milestone.

What it does:

1. Updates `PROJECT.md` with new milestone objectives
2. Runs fresh requirements gathering
3. Creates new `ROADMAP.md` for the milestone
4. Provides the planning entry point for Phase 1

Deliverables:

- Updated `.planning/PROJECT.md`
- New `.planning/ROADMAP.md`
- New `.planning/requirements/REQUIREMENTS.md`

`/gsd:discuss-phase N`

Purpose: Gather context and clarify design decisions before planning a phase.

When to use: Before `/gsd:plan-phase N` when you want to capture design preferences, approach decisions, or constraints specific to the phase.

What it does:

1. Asks adaptive questions about the phase — one at a time
2. Explores trade-offs and your preferences
3. Records all decisions in `N-CONTEXT.md`
4. This CONTEXT.md is then consumed by the planner — ensuring plans reflect your actual intentions

Parameters: `N` = phase number

Deliverables: `.planning/phases/NN-slug/NN-CONTEXT.md`

Why it matters: Plans without context are written from Claude's default assumptions. Plans with CONTEXT.md are written from your stated preferences. The difference is significant for complex phases.

`/gsd:plan-phase N`

Purpose: Create detailed, executable PLAN.md files for a phase.

When to use: After `/gsd:discuss-phase N` (or instead of it if you want to skip discussion).

What it does:

1. Validates phase exists in ROADMAP.md
2. Checks for and loads CONTEXT.md (asks if missing)
3. **Optionally runs `gsd-phase-researcher`** to research technical approach
4. **Spawns `gsd-planner`** — creates wave-grouped PLAN.md files with XML tasks
5. **Spawns `gsd-plan-checker`** — verifies plans will achieve the phase goal
6. **Revision loop** (up to 3 iterations) if checker finds issues
7. Presents final plan count + wave structure

Parameters:

- `N` — phase number (integer or decimal like `4.1`)
- `--research` — force re-research even if RESEARCH.md exists
- `--skip-research` — never research, plan directly
- `--gaps` — create gap-closure plans from VERIFICATION.md issues
- `--skip-verify` — skip plan checker

- `--auto` — auto-advance to execute-phase after planning

Deliverables:

- `.planning/phases/NN-slug/NN-RESEARCH.md`
- `.planning/phases/NN-slug/NN-01-PLAN.md`, `NN-02-PLAN.md`, etc.

`/gsd:execute-phase N`

Purpose: Execute all plans in a phase using parallel executor subagents.

When to use: After `/gsd:plan-phase N` produces PLAN.md files.

What it does:

1. Discovers all PLAN.md files, skips ones with matching SUMMARY.md (already done)
2. Groups plans into waves by `depends_on` frontmatter
3. For each wave, spawns parallel `gsd-executor` agents (one per plan)
4. Each executor: reads plan → executes tasks → commits atomically → writes SUMMARY.md → updates STATE.md
5. Spot-checks each executor's claims (verifies files exist, commits present)
6. Handles checkpoints (autonomous: false plans) — pauses for user input
7. After all waves, spawns `gsd-verifier` to check goal achievement
8. On success: updates ROADMAP.md and STATE.md

Parameters:

- `N` — phase number
- `--gaps-only` — execute only gap-closure plans
- `--auto` — auto-advance to next phase after completion

Deliverables:

- `.planning/phases/NN-slug/NN-01-SUMMARY.md` (per plan)
- `.planning/phases/NN-slug/NN-VERIFICATION.md`
- Git commits per task (atomic)

`/gsd:quick [description]`

Purpose: Execute a quick, bounded task with GSD guarantees but without optional agents.

When to use: Small fixes, polish items, or isolated improvements that don't need full phase planning.

What it does:

1. Creates a plan directory in `.planning/quick/N-task-slug/`
2. Writes a PLAN.md (compressed format, no research/checker agents)
3. Executes with atomic commits
4. Writes SUMMARY.md
5. Updates STATE.md

Parameters: Free-text description of the task.

Deliverables:

- `.planning/quick/N-task-slug/N-PLAN.md`
- `.planning/quick/N-task-slug/N-SUMMARY.md`
- Git commits

`/gsd:verify-work N`

Purpose: Validate built features through conversational user acceptance testing (UAT).

When to use: After completing a phase or before marking it complete — especially for UI features or user-facing behavior that automated tests can't cover.

What it does:

1. Checks for existing UAT sessions (resumable)
2. Presents one test scenario at a time: "Here's what should happen — does it?"
3. Records pass/fail for each scenario
4. Infers severity from user descriptions of failures
5. On completion, writes `NN-UAT.md` with test results and gap list
6. Gaps automatically feed into `/gsd:plan-phase N --gaps`

Parameters: `N` = phase number

Deliverables: `.planning/phases/NN-slug/NN-UAT.md`

`/gsd:plan-milestone-gaps`

Purpose: Create phases to close all gaps identified by a milestone audit.

When to use: After `/gsd:audit-milestone` reveals unmet requirements.

What it does: Creates gap-closure phases (e.g., `4.1`, `4.2`) for each gap cluster found in the milestone audit, then offers to execute them.

`/gsd:audit-milestone`

Purpose: Audit milestone completion against its original intent before archiving.

When to use: When all phases appear complete but before running `/gsd:complete-milestone`.

What it does: Reads original REQUIREMENTS.md and all VERIFICATION.md files to determine if every requirement was actually delivered. Produces a `vX.X-MILESTONE-AUDIT.md`.

Deliverables: `.planning/milestones/vX.X-MILESTONE-AUDIT.md`

`/gsd:complete-milestone`

Purpose: Archive a completed milestone and prepare for the next.

When to use: After `/gsd:audit-milestone` confirms all requirements are met.

What it does:

1. Moves all current phase files to `.planning/milestones/vX.X-phases/`
2. Archives ROADMAP.md and REQUIREMENTS.md to milestone directory
3. Removes active ROADMAP.md (signals between-milestone state to `/gsd:progress`)
4. Updates MILESTONES.md with completion date

`/gsd:debug [description]`

Purpose: Systematic debugging with persistent state across context resets.

When to use: Any bug, unexpected behavior, or test failure.

What it does:

1. Creates a debug session file in `.planning/debug/slug.md`
2. Spawns `gsd-debugger` agent using the 4-phase systematic debugging process
3. Records hypotheses tested, evidence gathered, root cause found
4. Debug session survives `/clear` — resume with `/gsd:debug`
5. On resolution, moves to `.planning/debug/resolved/`

Parameters: Free-text description of the bug.

Deliverables: `.planning/debug/slug.md`

`/gsd:map-codebase`

Purpose: Analyze existing codebase and produce structured documentation.

When to use: On brownfield projects, or before planning a phase that touches unfamiliar code.

What it does: Spawns 4 parallel `gsd-codebase-mapper` agents focused on:

- Technology stack and dependencies
- Architecture and module structure
- Code quality and patterns
- Potential concerns and tech debt

Deliverables: `.planning/codebase/TECH.md`, `ARCH.md`, `QUALITY.md`, `CONCERNS.md`

`/gsd:add-phase [description]`

Purpose: Add a new phase to the end of the current milestone's roadmap.

When to use: When scope expands or a new requirement needs a dedicated phase.

Parameters: Free-text phase description.

Deliverables: Updated `ROADMAP.md` with new phase entry.

`/gsd:insert-phase [N] [description]`

Purpose: Insert an urgent phase between existing phases using decimal numbering (e.g., `4.1` between 4 and 5).

When to use: When a gap closure or urgent fix needs its own phase without renumbering all subsequent phases.

`/gsd:remove-phase N`

Purpose: Remove a future phase from the roadmap and renumber subsequent phases.

When to use: When a planned phase is no longer needed.

`/gsd:list-phase-assumptions N`

Purpose: Surface Claude's assumptions about how a phase would be approached before planning.

When to use: Before `/gsd:plan-phase N` to catch misaligned assumptions early — especially on complex or ambiguous phases.

`/gsd:add-todo [description]`

Purpose: Capture an idea or task as a todo in STATE.md from current conversation context.

When to use: When you notice something that needs doing but don't want to interrupt current work.

`/gsd:check-todos`

Purpose: List all pending todos and select one to work on.

When to use: When you want to review and act on accumulated todos.

`/gsd:pause-work`

Purpose: Create a context handoff file when pausing work mid-phase.

When to use: When stopping for the day mid-task — creates a resume file so the next session can pick up exactly where you left off.

Deliverables: `.planning/RESUME.md`

`/gsd:resume-work`

Purpose: Resume work from a previous session with full context restoration.

When to use: When returning to work and `.planning/RESUME.md` exists (shown by `/gsd:progress`).

`/gsd:set-profile [quality|balanced|budget]`

Purpose: Switch the model profile used for GSD agents.

Profiles:

- `quality` — uses most capable models for all agents (slowest, most thorough)
- `balanced` — uses capable models for planning, faster for execution (default)
- `budget` — uses fastest models throughout (quickest, least thorough)

`/gsd:settings`

Purpose: Configure GSD workflow toggles interactively.

Configurable options: parallelization, auto-advance, commit behavior, nyquist validation, research enabled, plan checker enabled.

`/gsd:health`

Purpose: Diagnose planning directory health and optionally repair issues.

When to use: If planning files seem corrupted, inconsistent, or if commands fail unexpectedly.

`/gsd:cleanup`

Purpose: Archive accumulated phase directories from completed milestones.

When to use: When `.planning/phases/` has old completed phase directories from prior milestones cluttering the view.

`/gsd:update`

Purpose: Update GSD to the latest version with changelog display.

`/gsd:reapply-patches`

Purpose: Reapply local modifications (customizations) after a GSD update.

`/gsd:help`

Purpose: Show all available GSD commands and usage guide.

2.5 GSD Workflow

The full project lifecycle through GSD:

```
Phase 0: Project Setup
  /gsd:new-project          → Creates PROJECT.md + ROADMAP.md +
REQUIREMENTS.md

For each Phase:
  /gsd:progress             → Check state, get routing
  /gsd:discuss-phase N      → Capture design decisions → CONTEXT.md
  /gsd:plan-phase N         → Research + Plan + Verify → PLAN.md files
  /gsd:execute-phase N      → Parallel execution → SUMMARY.md + commits
  /gsd:verify-work N        → UAT testing → UAT.md
  (if gaps) /gsd:plan-phase N --gaps → Gap plans
  (if gaps) /gsd:execute-phase N --gaps-only → Fix gaps

Phase Complete:
  /gsd:audit-milestone      → Requirements coverage check
  /gsd:complete-milestone   → Archive + prepare next

Auto-advance option:
  /gsd:plan-phase N --auto  → Plan then immediately execute, then
transition
```

State transition between sessions:

```
Session ends → STATE.md updated with position + decisions
New session → /gsd:progress reads STATE.md → no context needed
```

2.6 Internal Subagents

GSD orchestrators spawn these specialized subagents:

Agent	Role	Input	Output
`gsd-phase-researcher`	Technical research for a phase	CONTEXT.md, REQUIREMENTS.md, phase description	`NN-RESEARCH.md`
`gsd-planner`	Creates executable PLAN.md files	RESEARCH.md, CONTEXT.md, ROADMAP.md	`NN-01-PLAN.md`, etc.

Agent	Role	Input	Output
`gsd-plan-checker`	Verifies plans will achieve phase goal	All PLAN.md files + phase goal	Pass/Issues report
`gsd-executor`	Executes a single plan atomically	PLAN.md + STATE.md	`NN-01-SUMMARY.md` + git commits
`gsd-verifier`	Goal-backward verification	All SUMMARY.md + must_haves	`NN-VERIFICATION.md`
`gsd-codebase-mapper`	Maps existing codebase	Source files	`TECH.md`, `ARCH.md`, etc.
`gsd-debugger`	Systematic debugging	Bug description + debug session	Updated debug session file
`gsd-integration-checker`	Cross-phase integration check	Multiple VERIFICATION.md files	Integration report
`gsd-roadmapper`	Creates milestone roadmaps	REQUIREMENTS.md + research	`ROADMAP.md`
`gsd-project-researcher`	Domain research for new projects	Project idea	Research files

3. Superpowers Framework

3.1 What Is Superpowers?

Superpowers is a set of coding discipline skills for Claude Code. Each skill is a protocol enforcing a specific engineering practice — brainstorming before building, TDD before implementation, verification before claiming completion.

Superpowers is installed at `~/.claude/skills/` and invoked via the `Skill` tool with `superpowers:skill-name`.

Philosophy: Skills enforce discipline that Claude might otherwise skip under time pressure or rationalization. Every skill has an "Iron Law" — a non-negotiable rule that cannot be bypassed.

Session scope: Superpowers skills provide discipline for the current session. They do not persist state across sessions (unlike GSD). However, their *outputs* (design docs, plans, implementation) become persistent artifacts.

3.2 Core Concepts

The Invocation Rule

> If there is even a 1% chance a skill applies to your task, you MUST invoke it before doing anything — including asking clarifying questions.

This rule prevents rationalization ("too simple to need a skill") from degrading code quality.

Iron Laws

Each skill has a non-negotiable rule:

- TDD: "No production code without a failing test first"
- Debugging: "No fixes without root cause investigation first"
- Verification: "No completion claims without fresh verification evidence"
- Brainstorming: "No implementation until design is presented and approved"

Skill Type: Rigid vs. Flexible

- **Rigid** (TDD, Debugging, Verification): Follow exactly. Adapting away from the ritual defeats the purpose.
- **Flexible** (Brainstorming, Writing Plans): Adapt principles to context.

3.3 All Skills

`superpowers:using-superpowers`

Purpose: Establish how to find and use skills at the start of every conversation.

When invoked: Automatically at session start (via system reminder).

Iron Law: Skill check happens BEFORE any response, including clarifying questions.

What it provides:

- Decision graph for when to invoke skills
- Red flag list (common rationalizations to avoid)
- Skill priority order: process skills first, implementation skills second

---

`superpowers:brainstorming`

Purpose: Turn ideas into fully formed designs through collaborative dialogue before any implementation begins.

When to use: Before creating features, building components, adding functionality, or modifying behavior — ANY creative work.

Iron Law / Hard Gate: Do NOT write any code, invoke any implementation skill, or take any implementation action until a design has been presented AND the user has approved it.

Process (sequential):

1. **Explore project context** — read files, docs, recent commits
2. **Ask clarifying questions** — one at a time, understand purpose/constraints/success criteria
3. **Propose 2-3 approaches** — with trade-offs and recommendation

4. **Present design sections** — get approval after each section
5. **Write design doc** — save to `docs/plans/YYYY-MM-DD-<topic>-design.md` and commit
6. **Transition to writing-plans** — the ONLY skill invoked from brainstorming

Parameters: None — conversational.

Deliverables: `docs/plans/YYYY-MM-DD-<topic>-design.md`

Key principles:

- One question at a time (not a list)
- Multiple choice preferred over open-ended
- YAGNI ruthlessly — remove unnecessary features from all designs
- Always propose 2-3 alternatives before settling

`superpowers:writing-plans`

Purpose: Create comprehensive, bite-sized implementation plans from a design spec.

When to use: After brainstorming produces an approved design. Before touching any code.

Process:

1. Announce: "I'm using the writing-plans skill to create the implementation plan."
2. Write the plan with exact file paths, complete code snippets, and exact commands
3. Each step is 2-5 minutes of work (RED → verify RED → GREEN → verify GREEN → REFACTOR → commit)
4. Save to `docs/plans/YYYY-MM-DD-<feature-name>.md`
5. Offer two execution modes: Subagent-Driven (same session) or Parallel Session (new session)

Plan format per task:

```
### Task N: [Component Name]

**Files:**
- Create: `exact/path/to/file.py`
- Modify: `exact/path/to/existing.py:123-145`
- Test: `tests/exact/path/to/test.py`

**Step 1: Write the failing test** [code]
**Step 2: Run test to verify it fails** [command + expected output]
**Step 3: Write minimal implementation** [code]
**Step 4: Run test to verify it passes** [command + expected output]
**Step 5: Commit** [git command]
```

Deliverables: `docs/plans/YYYY-MM-DD-<feature-name>.md`

`superpowers:test-driven-development`

Purpose: Enforce Red-Green-Refactor cycle before writing any production code.

When to use: Implementing any feature, bugfix, or behavior change.

Iron Law: "No production code without a failing test first."

The cycle:

```
RED: Write one failing test (one behavior, clear name, real code)
  ↓ VERIFY RED: Run test — must fail for the right reason
GREEN: Write minimal code to pass (no extras, no refactoring)
  ↓ VERIFY GREEN: Run test — must pass, no other tests break
REFACTOR: Clean up (no behavior changes, stay green)
  ↓ Repeat
```

Non-negotiable rules:

- Code before test? Delete it. Start over.
- Test passes immediately? You're testing existing behavior. Fix the test.
- "I'll write tests after" — passes immediately, proves nothing

Good test qualities:

- One behavior per test
- Name describes behavior (not "test1")
- Tests real code (not mocks unless unavoidable)
- Fails for the right reason

`superpowers:systematic-debugging`

Purpose: Enforce root cause investigation before proposing any fix.

When to use: Any bug, test failure, unexpected behavior, performance problem, build failure.

Iron Law: "No fixes without root cause investigation first."

Four phases:

1. **Root Cause Investigation**
    - Read error messages carefully (stack traces, line numbers)
    - Reproduce consistently (can you trigger it reliably?)
    - Check recent changes (git diff, new dependencies)
    - In multi-component systems: add diagnostic instrumentation at each boundary, run once to find WHERE it breaks
    - Trace data flow backward (where does bad value originate?)
2. **Pattern Analysis**
    - Find working examples of similar code

    - Compare against references (read completely, not skimming)
    - Identify every difference between working and broken
    - Understand all dependencies and assumptions

3. **Hypothesis and Testing**

    - State one clear hypothesis: "I think X is root cause because Y"
    - Make smallest possible change to test it
    - One variable at a time
    - If wrong: form new hypothesis, don't pile on more fixes

4. **Implementation**

    - Create failing test case BEFORE fixing
    - Implement single fix for root cause
    - Verify fix (test passes, no regressions)
    - If 3+ fixes fail: question architecture, stop and discuss

Rule after 3 failed fixes: Do NOT attempt a 4th fix. The architecture may be wrong. Discuss with user.

`superpowers:verification-before-completion`

Purpose: Require actual evidence before claiming work is complete.

When to use: Before claiming tests pass, build succeeds, bug is fixed, requirements met, or before committing/creating PRs.

Iron Law: "No completion claims without fresh verification evidence."

The gate function:

```
1. IDENTIFY: What command proves this claim?
2. RUN: Execute the FULL command (fresh, not cached)
3. READ: Full output, check exit code, count failures
4. VERIFY: Does output confirm the claim?
5. ONLY THEN: Make the claim
```

What counts as evidence:

- "Tests pass" → `pytest -q` output showing 0 failures (just run)
- "Build succeeds" → build command exit 0 (not just "linter passed")
- "Bug fixed" → original symptom test passes (not just code changed)
- "Requirements met" → line-by-line checklist against spec

Red flags:

- "Should work now" / "Probably" / "Seems to"
- Expressing satisfaction before running verification
- Trusting agent success reports without independent check

`superpowers:dispatching-parallel-agents`

Purpose: Handle multiple independent problems concurrently using specialized subagents.

When to use: 2+ independent tasks — different test files failing, different subsystems broken, different features to implement simultaneously.

Process:

1. Identify independent domains (can each be worked on without context from others?)
2. Create focused agent prompts (specific scope, clear goal, constraints, expected output)
3. Dispatch all agents in parallel (one `Task` call per agent in same message)
4. Review all returns together — check for conflicts
5. Run full suite to verify all fixes work together

Good agent prompt structure:

- Specific scope (one file or subsystem)
- Clear goal (make these 3 tests pass)
- Constraints (don't change other code)
- Expected output (return root cause + what changed)

Do NOT use when:

- Failures are related (fixing one might fix others)
- Agents would edit the same files
- Need full system context to understand the problem

`superpowers:subagent-driven-development`

Purpose: Execute an implementation plan task-by-task within the current session using fresh subagents with two-stage review.

When to use: Have a written plan, tasks are mostly independent, want to stay in current session.

Process per task:

1. Dispatch implementer subagent with full task text + context
2. Answer any questions subagent raises
3. Subagent implements, tests, commits, self-reviews
4. Dispatch spec compliance reviewer — checks code matches spec exactly
5. If issues: implementer fixes, reviewer re-reviews
6. Dispatch code quality reviewer — checks implementation quality
7. If issues: implementer fixes, reviewer re-reviews
8. Mark task complete
9. Repeat for next task

After all tasks:

- Dispatch final code reviewer for entire implementation
- Use `superpowers:finishing-a-development-branch`

Red flags:

- Skip spec review before code quality review (wrong order)
- Let implementer self-review replace actual review
- Move to next task while issues are open

`superpowers:executing-plans`

Purpose: Execute a written plan in a separate session with batch execution and checkpoint reviews.

When to use: Have a written plan, want parallel session execution (not same session), need human-in-loop between batches.

Process:

1. Load and critically review plan — raise concerns before starting
2. Create TodoWrite
3. Execute first 3 tasks as a batch
4. Report: what was implemented + verification output + "Ready for feedback"
5. Apply feedback if needed
6. Execute next batch
7. Repeat until complete
8. Use `superpowers:finishing-a-development-branch`

Difference from subagent-driven-development:

- Different session (requires handoff)
- Batch execution (not per-task)
- Human reviews between batches (not after each task)

`superpowers:using-git-worktrees`

Purpose: Create isolated git worktrees for feature work to avoid polluting the main workspace.

When to use: Before executing any implementation plan, before starting feature work that needs isolation.

Process:

1. Check current branch and working tree status
2. Select appropriate parent directory
3. Create worktree: `git worktree add ../<branch-name> -b <branch-name>`
4. Verify worktree is clean and correct
5. All implementation work happens in the worktree

Why it matters: Worktrees allow parallel work on different features without branch switching. Each worktree has its own working directory and index.

`superpowers:requesting-code-review`

Purpose: Request thorough code review after completing implementation.

When to use: After completing significant features, implementing major changes, or before merging.

What it ensures: Security review, architecture alignment, test coverage, code quality, and requirement compliance are all checked before integration.

`superpowers:receiving-code-review`

Purpose: Process code review feedback with technical rigor before implementing suggestions.

When to use: Upon receiving code review comments — especially if feedback seems unclear or technically questionable.

Iron Law: Do not implement suggestions blindly. Verify technical correctness first.

Process:

1. Read all feedback before implementing anything
2. For each suggestion: understand WHY, verify it's correct, check for conflicts with other suggestions
3. Question suggestions that seem wrong — reviewer may have incomplete context
4. Implement verified improvements
5. Re-verify complete system after changes

`superpowers:finishing-a-development-branch`

Purpose: Guide completion of development work after all tasks are done and tests pass.

When to use: After all implementation tasks complete and full test suite passes.

Process:

1. Verify tests pass (do NOT proceed if failing)
2. Determine base branch
3. Present 4 options: Merge locally | Push & create PR | Keep as-is | Discard
4. Execute chosen option
5. Cleanup worktree (for options 1 and 4)

Never: Force-push, merge without test verification, delete without confirmation.

`superpowers:writing-skills`

Purpose: Create, edit, or verify Superpowers skills before deployment.

When to use: Creating new skills, editing existing skills, or testing that skills work correctly.

3.4 Superpowers Workflow

The typical Superpowers skill invocation chain for a new feature:

```
New feature request
  ↓
superpowers:brainstorming
  → Explores intent, proposes approaches, gets approval
  → Writes docs/plans/YYYY-MM-DD-design.md
  ↓
superpowers:writing-plans
  → Creates bite-sized TDD plan
  → Saves docs/plans/YYYY-MM-DD-feature.md
  ↓
superpowers:using-git-worktrees
  → Creates isolated workspace
  ↓
EITHER superpowers:subagent-driven-development (same session)
OR     superpowers:executing-plans (new session)
  → Each task uses superpowers:test-driven-development internally
  ↓
superpowers:verification-before-completion
  → Runs full test suite, confirms output
  ↓
superpowers:requesting-code-review
  → Review before merge
  ↓
superpowers:finishing-a-development-branch
  → Merge / PR / keep
```

For bugs:

```
Bug discovered
  ↓
superpowers:systematic-debugging
  → 4-phase investigation finds root cause
  ↓
superpowers:test-driven-development
  → Write failing test reproducing bug
  → Fix → verify green
  ↓
superpowers:verification-before-completion
  → Full suite still green
```

---

4. Combining Both Frameworks

4.1 Artifact Sharing Map

The two frameworks share artifacts through a well-defined boundary:

```
GSD produces:                      Superpowers consumes:
═══════════════════════════════════════════════════════

.planning/STATE.md          →      CLAUDE.md context (executor reads it)
.planning/NN-PLAN.md        →      gsd-executor (which runs TDD internally)
.planning/PROJECT.md        →      brainstorming (for project context)
.planning/NN-RESEARCH.md    →      writing-plans (technical context)
.planning/NN-CONTEXT.md     →      writing-plans (design decisions)

Superpowers produces:              GSD consumes:
═══════════════════════════════════════════════════════

docs/plans/design.md        →      gsd-planner reads for design intent
docs/plans/feature.md       →      gsd-planner can include in plan context
git commits                 →      gsd-executor spot-checks commits
test results                →      gsd-verifier checks must_haves
```

The Central Bridge: CLAUDE.md

`CLAUDE.md` is the shared contract between both frameworks:

- GSD executor agents read it for project-specific conventions
- Superpowers skills read it for project context
- Updates to `CLAUDE.md` propagate to all future agent invocations automatically

The Second Bridge: docs/plans/

Brainstorming produces `docs/plans/YYYY-MM-DD-design.md`. When `/gsd:plan-phase` runs, the `gsd-planner` can reference these design docs via the context files list, ensuring plans reflect design intent.

4.2 Decision Matrix: Which to Use When

Situation	Use GSD	Use Superpowers	Use Both
New project, zero context	`/gsd:new-project`	—	GSD creates context Superpowers needs
"What should I work on?"	`/gsd:progress`	—	GSD routes, Superpowers then executes
Designing a new feature	—	`brainstorming`	Brainstorming output feeds GSD planner
Planning a full phase	`/gsd:plan-phase`	—	GSD orchestrates research+planning agents
Executing 1 plan	—	`executing-plans` or `subagent-driven-development`	Superpowers for discipline

Situation	Use GSD	Use Superpowers	Use Both
Executing whole phase	`/gsd:execute-phase`	TDD inside executor	GSD orchestrates, TDD is enforced internally
Found a bug	`/gsd:debug`	`systematic-debugging`	Either; GSD adds persistence
Implementation task	—	`tdd` + `verification`	Superpowers enforces quality
Multiple parallel tasks	`/gsd:execute-phase` (parallel waves)	`dispatching-parallel-agents`	Depends on scope
Phase complete, verify it	`/gsd:verify-work`	`verification-before-completion`	Both: UAT + automated checks
Before merging	—	`requesting-code-review` + `finishing-a-development-branch`	Superpowers handles merge

4.3 Master Combined Workflow

This is the recommended workflow when starting a new feature or phase, combining both frameworks for maximum quality:

```
┌──────────────────────────────────────────────────────────────────┐
│  SESSION 1: Setup & Design (Superpowers-led)                     │
│                                                                   |
│  1. /gsd:progress                                                 |
│     → Get current position, confirm this is the right phase      |
│                                                                   |
│  2. superpowers:brainstorming                                     |
│     → Explore intent, propose approaches, get design approval    |
│     → Write docs/plans/YYYY-MM-DD-design.md                      |
│     → Commit design doc                                           |
│                                                                   |
│  3. /gsd:discuss-phase N                                          |
│     → Record design decisions as .planning/NN-CONTEXT.md         |
│     → CONTEXT.md references design doc for full detail            |
└──────────────────────────────────────────────────────────────────┘
                              ↓ /clear
┌──────────────────────────────────────────────────────────────────┐
│  SESSION 2: Planning (GSD-led)                                    |
│                                                                   |
│  4. /gsd:plan-phase N                                             |
│     → gsd-phase-researcher: technical research                    |
│       (reads CONTEXT.md → picks up design decisions)             |
│     → gsd-planner: creates PLAN.md files                          |
```

```
│       (reads RESEARCH.md + CONTEXT.md + design doc)              │
│     → gsd-plan-checker: verifies plans will achieve goal        │
│     → Revision loop if needed                                   │
└─────────────────────────────────────────────────────────────────┘
                              ↓ /clear
┌─────────────────────────────────────────────────────────────────┐
│  SESSION 3: Execution (Both frameworks)                         │
│                                                                  │
│  5. superpowers:using-git-worktrees                             │
│     → Create isolated branch for the work                       │
│                                                                  │
│  Option A — Full phase, parallel execution:                     │
│    /gsd:execute-phase N                                         │
│    → gsd-executor agents read PLAN.md files                     │
│    → Each executor follows TDD (enforced via CLAUDE.md)         │
│    → Atomic commits per task                                     │
│    → SUMMARY.md per plan                                        │
│    → gsd-verifier checks phase goal achievement                 │
│                                                                  │
│  Option B — Single plan, same-session quality control:          │
│    superpowers:subagent-driven-development                      │
│    → Reads docs/plans/feature.md                                │
│    → Fresh subagent per task                                    │
│    → Spec review + code quality review per task                 │
│    → superpowers:finishing-a-development-branch at end          │
└─────────────────────────────────────────────────────────────────┘
                              ↓ /clear
┌─────────────────────────────────────────────────────────────────┐
│  SESSION 4: Validation (Both frameworks)                        │
│                                                                  │
│  6. superpowers:verification-before-completion                  │
│     → Run full test suite, confirm output                       │
│     → Evidence before any completion claims                     │
│                                                                  │
│  7. /gsd:verify-work N                                          │
│     → Conversational UAT for user-facing behavior               │
│     → Records pass/fail, creates NN-UAT.md                     |
│                                                                  │
│  8. superpowers:requesting-code-review                          │
│     → Final code review before merge                            │
│                                                                  │
│  9. superpowers:finishing-a-development-branch                  │
│     → Merge or PR                                               │
│                                                                  │
│  10. /gsd:progress                                              │
│      → Confirm completion, get routing to next phase            │
└─────────────────────────────────────────────────────────────────┘
```

4.4 Context Continuity Protocol

The most critical challenge when combining frameworks is ensuring no context is lost across sessions. Here's the protocol:

What GSD Stores (Persists Across Sessions)

File	What it preserves
`STATE.md`	Current position, all decisions made, blockers, todos
`ROADMAP.md`	Phase completion status, goals, requirements coverage
`NN-CONTEXT.md`	User design decisions for phase N
`NN-RESEARCH.md`	Technical research findings for phase N
`NN-PLAN.md`	The complete executable plan with all tasks
`NN-SUMMARY.md`	What was built, files created, decisions made
`NN-VERIFICATION.md`	What was verified, what gaps remain

What Superpowers Stores

File	What it preserves
`docs/plans/YYYY-MM-DD-design.md`	Design decisions and architecture
`docs/plans/YYYY-MM-DD-feature.md`	Implementation plan with tasks
Git history	Every atomic commit with descriptive messages

The `/clear` Handoff Protocol

Before every `/clear`, ensure:

1. **GSD STATE.md is updated** — executors do this automatically. If you made decisions manually, add them: `/gsd:add-todo` or edit STATE.md directly.
2. **Design docs are committed** — `docs/plans/design.md` should be in git before clearing.
3. **CONTEXT.md exists** — if you discussed design in the session, run `/gsd:discuss-phase N` to record it formally.
4. **Pausing mid-task** — run `/gsd:pause-work` to create a RESUME.md with exact stopping point.
5. **Next session start** — always run `/gsd:progress` first to reload full context.

Artifact Flow Example (Full Blitz AgentOS Feature)

```
Brainstorming produces:
  docs/plans/2026-02-27-workflow-crud-design.md
  (contains: why React Flow, how definition_json is structured, API design)
            ↓
Discuss-phase records:
  .planning/phases/04-canvas/04-CONTEXT.md
  (contains: references design doc, adds: owner_user_id must be nullable)
```

```
            ↓
Research produces:
  .planning/phases/04-canvas/04-RESEARCH.md
  (contains: React Flow v12 API, LangGraph StateGraph patterns)
            ↓
Planner reads all three → produces:
  .planning/phases/04-canvas/04-01-PLAN.md
  (contains: 6 tasks, must_haves, wave: 1)
            ↓
Executor reads plan + CLAUDE.md → produces:
  git commits: feat(04-01): add workflow SQLAlchemy models
               feat(04-01): create Alembic migration 009
               feat(04-01): add workflow CRUD endpoints
  .planning/phases/04-canvas/04-01-SUMMARY.md
  (contains: files created, decision: owner_user_id nullable, 15 tests
added)
            ↓
Verifier reads SUMMARY + must_haves → produces:
  .planning/phases/04-canvas/04-VERIFICATION.md
  (status: passed, score: 8/8 must_haves verified)
            ↓
STATE.md updated:
  position: Phase 4, Plan 01 complete
  decision: [04-01] owner_user_id nullable (no FK constraint per project
rules)
```

Every decision made in one session is readable by agents in the next session. No context is ever truly lost — it's all in the files.

---

5. Quick Reference

GSD Commands Cheat Sheet

```
# Navigation
/gsd:progress                        # Where am I? What's next?
/gsd:help                            # List all commands

# Project lifecycle
/gsd:new-project                     # Initialize project
/gsd:new-milestone                   # Start new milestone

# Phase workflow
/gsd:discuss-phase 4                 # Gather design context for Phase 4
/gsd:list-phase-assumptions 4        # See Claude's assumptions before
planning
/gsd:plan-phase 4                    # Plan Phase 4 (research + plan +
verify)
/gsd:plan-phase 4 --auto             # Plan then immediately execute
/gsd:plan-phase 4 --gaps             # Plan fixes for gaps in
VERIFICATION.md
```

```
/gsd:execute-phase 4                  # Execute all Phase 4 plans
/gsd:execute-phase 4 --gaps-only      # Execute only gap-closure plans
/gsd:verify-work 4                    # UAT testing for Phase 4

# Quick tasks
/gsd:quick fix owner_user_id nullable # Quick fix with GSD guarantees

# Roadmap management
/gsd:add-phase "Phase 6: Observability"
/gsd:insert-phase 4 "HITL fix"        # Creates Phase 4.1
/gsd:remove-phase 7                   # Remove phase

# Milestone management
/gsd:audit-milestone                  # Requirements coverage check
/gsd:complete-milestone               # Archive and prepare next
/gsd:cleanup                          # Archive old phase directories

# Debugging
/gsd:debug "HITL amber ring not showing"

# Session management
/gsd:pause-work                       # Create RESUME.md before stopping
/gsd:resume-work                      # Resume from RESUME.md
/gsd:add-todo "Start WhatsApp verification"
/gsd:check-todos                      # Review todos

# Configuration
/gsd:set-profile quality              # Use most capable models
/gsd:settings                         # Configure GSD options
/gsd:health                           # Diagnose planning directory
/gsd:update                           # Update GSD version
```

Superpowers Skills Cheat Sheet

```
superpowers:brainstorming             # Before ANY feature implementation
superpowers:writing-plans             # After brainstorming, before coding
superpowers:test-driven-development   # Before writing implementation code
superpowers:systematic-debugging      # Before proposing any fix
superpowers:verification-before-completion  # Before claiming work is done
superpowers:dispatching-parallel-agents     # For 2+ independent tasks
superpowers:subagent-driven-development     # Execute plan (same session)
superpowers:executing-plans           # Execute plan (parallel session)
superpowers:using-git-worktrees       # Before implementation begins
superpowers:finishing-a-development-branch  # After all tasks complete
superpowers:requesting-code-review    # Before merging
superpowers:receiving-code-review     # When reviewing feedback
```

Workflow Decision Guide

```
```

```
Q: Starting a new session?
→ /gsd:progress

Q: New feature to build?
→ superpowers:brainstorming → superpowers:writing-plans → /gsd:discuss-
phase N

Q: Phase needs planning?
→ /gsd:plan-phase N

Q: Plans exist, ready to execute?
→ /gsd:execute-phase N (full phase) OR superpowers:subagent-driven-
development (one plan)

Q: Found a bug?
→ superpowers:systematic-debugging (session) OR /gsd:debug (persistent)

Q: About to claim work is done?
→ superpowers:verification-before-completion first

Q: Phase complete, verify it?
→ superpowers:verification-before-completion + /gsd:verify-work N

Q: Ready to merge?
→ superpowers:requesting-code-review → superpowers:finishing-a-development-
branch

Q: Gaps found in verification?
→ /gsd:plan-phase N --gaps → /gsd:execute-phase N --gaps-only

Q: All phases complete?
→ /gsd:audit-milestone → /gsd:complete-milestone
```

Key File Locations

```
# GSD (persists across sessions)
.planning/STATE.md                    # Current project state
.planning/ROADMAP.md                  # Phase plan with completion
.planning/PROJECT.md                  # Project identity and decisions
.planning/phases/NN-slug/NN-PLAN.md  # Executable plans
.planning/phases/NN-slug/NN-SUMMARY.md  # What was built

# Superpowers (session outputs)
docs/plans/YYYY-MM-DD-design.md      # Design decisions
docs/plans/YYYY-MM-DD-feature.md     # Implementation plan

# Shared bridge
CLAUDE.md                            # Project conventions (read by all
agents)
```

Last updated: 2026-02-27 Applies to: Blitz AgentOS project using GSD v2+ and Superpowers v1.x

6. Appendix: Optimizing CLAUDE.md for Both Frameworks

6.1 Why CLAUDE.md Is the Central Bridge

`CLAUDE.md` is the **only file guaranteed to be read by every agent in every context** — raw Claude sessions, GSD executor subagents, Superpowers skill implementations, and mid-phase resumptions all start from it. This makes it uniquely powerful as a shared contract.

```
Every agent in every session reads:
                  CLAUDE.md
                 /          \
         GSD agents      Superpowers sessions
        /     |    \          |          \
  planner  executor verifier brainstorm   tdd
                              |
                         raw Claude
```

What CLAUDE.md currently does well:

- Establishes the mandatory `docs/dev-context.md` read protocol
- Provides clean DO/DON'T tables for common decisions
- Documents technology stack, coding standards, and architecture invariants
- Has a Section 12 introducing both frameworks

What CLAUDE.md currently lacks (causing repeated rediscovery across sessions):

Gap	Impact	How Often Rediscovered
No canonical verification commands	Agents guess wrong test commands, hit uv timeouts	Every new session
No decision recording protocol	Decisions made during execution may not reach STATE.md	~30% of plans
Critical gotchas in STATE.md only	Fresh raw sessions (not via /gsd:progress) miss them	Any non-GSD session
Thin GSD+Superpowers section	Agents don't know mandatory invocation rule or artifact protocol	Every session start
No commit format guide	GSD spot-checks by grep pattern — wrong format breaks tracking	~20% of commits
No migration chain documentation	New agents guess the next Alembic revision number	Every migration

Gap	Impact	How Often Rediscovered
No test baseline count	No signal when a commit inadvertently drops test coverage	Silently
No context continuity protocol	`/clear` protocol not known; STATE.md may not be updated	After every /clear
No docs output location convention	Design docs end up in random places	During brainstorming

6.2 Analysis of Current CLAUDE.md

Strengths

AGENT INSTRUCTIONS (top of file): The mandatory `docs/dev-context.md` read with confirmation statement is excellent — it enforces context loading before any action. This is the right pattern.

DO/DON'T tables: Scannable, actionable, and unambiguous. Agents read tables reliably.

Technology stack section: Comprehensive. Model aliases, port map, and package manager rules prevent common mistakes.

Architecture invariants (Section 7): The security gates, memory isolation, and single-entry-point rules are critical to enforce here — they're not repeated elsewhere.

Section 12 introduction: The table comparing GSD vs Superpowers is a good starting point.

Gaps (Detailed)

Gap 1 — No canonical verification commands

The most common agent mistake is running `uv run pytest` (times out on this machine) instead of `.venv/bin/pytest`. This is documented in STATE.md decisions but not in CLAUDE.md. Every new agent that doesn't route through `/gsd:progress` will make this mistake.

Gap 2 — No decision recording protocol

GSD executor agents make decisions ("owner_user_id must be nullable") but there's no instruction to record them in STATE.md. The format `- [04-01]: <decision> — <rationale>` in STATE.md Decisions section is the correct pattern, but agents don't know to do it.

Gap 3 — Critical gotchas in wrong location

The following are in STATE.md decisions but should also be in CLAUDE.md:

- `uv run` timeouts → use `.venv/bin/` directly
- Alembic migration from host needs `docker exec psql`
- Next.js 15 requires `Promise<{id: string}>` for async params
- FastAPI route ordering: literal paths before `/{uuid}` params

- Current migration chain head (what number is next)

Gap 4 — Mandatory Superpowers invocation is soft

Section 12 says "use BEFORE implementing" but doesn't state the Iron Law: "1% chance a skill applies = MUST invoke it." Agents rationalize skipping skills without this explicit rule.

Gap 5 — No commit format guide

GSD's spot-check verifies commits exist by grepping for `feat({phase}-{plan})` patterns. If agents commit with different formats (e.g., `add workflow models` without phase prefix), the spot-check may report false failures, triggering unnecessary retries.

Gap 6 — No Alembic migration chain state

Each new session that needs to create a migration guesses the revision number. The current chain (001 → 002 → 003 → merge → 004 → ... → 011) should be documented with the next expected number.

Gap 7 — Test baseline unknown

Without a documented baseline (currently 258 tests passing), agents don't know when to be alarmed. A plan that inadvertently drops 10 tests may not trigger warnings.

Gap 8 — Context continuity absent

No instruction for what to do before `/clear` (update STATE.md, commit design docs) or after (always start with `/gsd:progress`). This causes sessions to start without proper context.

6.3 Recommended Enhancements

The following enhancements are recommended, grouped by priority:

Priority 1 — Critical (causes repeated rediscovery)

Enhancement	Location in CLAUDE.md	Benefit
Canonical verification commands	New Section 13	Eliminates uv timeout issues
Decision recording protocol	AGENT INSTRUCTIONS Step 4	Decisions survive session boundaries
Critical gotchas quick reference	New Section 14	Eliminates repeated rediscovery
Commit format specification	Section 12 / New DO/DON'T row	GSD spot-checks work correctly

Priority 2 — High (improves framework integration)

Enhancement	Location in CLAUDE.md	Benefit

Enhancement	Location in CLAUDE.md	Benefit
Mandatory Superpowers invocation rule	Section 12	Skills actually used, not rationalized away
Artifact sharing protocol	Section 12	Agents know what files GSD/Superpowers exchange
Context continuity protocol	Section 12 / New Section 15	No context loss across /clear
Migration chain documentation	New Section 14	No guessing of revision numbers

Priority 3 — Good to have

Enhancement	Location in CLAUDE.md	Benefit
Test baseline count	New Section 13	Early warning on coverage drops
Design doc naming convention	Section 12	Consistent docs/plans/ output
docs/plans/ update in directory tree	Section 8	Agents know where to save design docs

---

6.4 Enhanced Sections (Drop-in Additions)

These are the exact content blocks recommended for addition/replacement in CLAUDE.md.

Enhancement A: Expand AGENT INSTRUCTIONS (replace Step 4)

Current Step 4:

```
**Step 4 — Update on discovery.** When you find a new endpoint, URL
mapping, or gotcha during work:
- Add it to `docs/dev-context.md` immediately (correct section + Update Log
at bottom).
- Never leave a discovery undocumented for the next session.
```

Recommended replacement:

```
**Step 4 — Update on discovery.** When you find a new endpoint, URL
mapping, or gotcha:
- Add it to `docs/dev-context.md` immediately (correct section + Update Log
at bottom).
- Never leave a discovery undocumented for the next session.
```

```
**Step 5 — Record decisions in STATE.md.** When you make a technical
decision during
implementation (e.g., "owner_user_id must be nullable", "use asyncio.run()
in Celery tasks"):
- Add to `.planning/STATE.md` → Decisions section with format:
  `- [Phase-Plan or context]: <decision> — <rationale>`
- Commit STATE.md update with the plan's final commit.
- This ensures decisions survive context resets and are visible to future
agents.

**Step 6 — Use canonical commands.** See Section 13 for the exact test,
build, and migration
commands. Do NOT guess commands — the wrong ones either time out or produce
misleading output.
```

Enhancement B: Add Commit Format to DO/DON'T

Add a new table after the existing "Package Management" table:

```
### Commits (GSD Tracking)
| DO | DON'T |
|----|-------|
| `feat(04-01): add workflow SQLAlchemy models` | `add workflow models` (no
phase prefix) |
| `fix(04-03): handle HITL interrupt correctly` | `fix bug` (too vague) |
| `docs(phase-04): update STATE.md position` | `update docs` |
| `test(04-01): add workflow model tests` | `add tests` |
| One commit per task, one task per commit | Bundle multiple tasks in one
commit |

**Format:** `<type>(<phase>-<plan>): <description>`
- `type`: feat / fix / test / docs / refactor / chore
- `phase-plan`: e.g., `04-01` (Phase 4, Plan 01) — omit for cross-cutting
changes
- `description`: imperative mood, lowercase, ≤72 chars
```

Enhancement C: Expand Section 12 (GSD + Superpowers Workflow)

Replace current Section 12 with this expanded version:

```
## 12. GSD + Superpowers Workflow

This project uses two complementary AI tool systems:

| Tool | Purpose | Persistence | Entry Point |
|------|---------|-------------|-------------|
| **GSD** | Project lifecycle: milestones → phases → plans → execution |
```

```
Across sessions via `.planning/` files | `/gsd:progress` |
| **Superpowers** | Session discipline: TDD, brainstorming, verification |
Within session (reads CLAUDE.md for context) | Skill tool: `superpowers:X`
|

### Mandatory Invocation Rule

> **If there is even a 1% chance a Superpowers skill applies to your task,
you MUST invoke
> it via the Skill tool BEFORE doing anything — including asking clarifying
questions.**

This is not optional. Rationalization patterns to ignore:
- "Too simple to need a skill" — simple code breaks too
- "I need context first" — skills tell you HOW to gather context
- "Just this once" — no exceptions

### Key GSD Commands
```

/gsd:progress → ALWAYS start here — loads state, routes to next action /gsd:discuss-phase N → Capture design decisions before planning /gsd:plan-phase N → Research + create PLAN.md files /gsd:execute-phase N → Parallel wave execution of all plans /gsd:verify-work N → UAT testing, creates UAT.md with gaps /gsd:quick → Quick fix with GSD guarantees (no research/checker) /gsd:debug → Systematic debug with persistent state

```
### Key Superpowers Skills
```

superpowers:brainstorming → BEFORE any feature implementation superpowers:writing-plans → After brainstorming, before coding superpowers:test-driven-development → BEFORE writing implementation code superpowers:systematic-debugging → BEFORE proposing any fix superpowers:verification-before-completion → BEFORE claiming work is done superpowers:dispatching-parallel-agents → For 2+ independent tasks superpowers:subagent-driven-development → Execute plan in current session superpowers:executing-plans → Execute plan in parallel session superpowers:using-git-worktrees → Before any implementation begins superpowers:finishing-a-development-branch → After all tasks complete

```
### Artifact Sharing Protocol

GSD and Superpowers share artifacts through these files — always produce
and consume
them in the correct direction:
```

Superpowers → GSD: docs/plans/YYYY-MM-DD-design.md → gsd-planner reads for design intent docs/plans/YYYY-MM-DD-plan.md → gsd-executor can execute git commits (atomic, per task) → gsd spot-checks for existence

GSD → Superpowers: .planning/PROJECT.md → brainstorming reads for project context .planning/NN-PLAN.md → subagent-driven-development executes .planning/NN-RESEARCH.md → writing-plans uses for technical context .planning/NN-CONTEXT.md → writing-plans uses for design decisions

```
### Context Continuity Protocol (Before/After /clear)

**Before every /clear:**
1. `STATE.md` has current position and all decisions made this session
2. Design docs (`docs/plans/`) are committed to git
3. If mid-task: run `/gsd:pause-work` to create `.planning/RESUME.md`
4. If decisions were made manually: add them to STATE.md Decisions section

**After /clear (new session):**
1. ALWAYS start with `/gsd:progress` — it reloads full context from files
2. Confirm position matches expectation before starting new work
3. If RESUME.md exists: run `/gsd:resume-work` instead

### Recommended Workflow per Feature
```

[Session 1 — Design] superpowers:brainstorming → docs/plans/YYYY-MM-DD-design.md /gsd:discuss-phase N → .planning/NN-CONTEXT.md

/clear

[Session 2 — Planning] /gsd:plan-phase N → .planning/NN-0X-PLAN.md files

/clear

[Session 3 — Execution] superpowers:using-git-worktrees → isolated branch /gsd:execute-phase N → SUMMARY.md + atomic commits (each task uses TDD internally — enforced via CLAUDE.md + .agents/skills/)

/clear

[Session 4 — Validation] superpowers:verification-before-completion → fresh test run evidence /gsd:verify-work N → UAT.md superpowers:requesting-code-review → code review superpowers:finishing-a-development-branch → merge or PR

/gsd:progress → confirm completion, get next phase

Enhancement D: New Section 13 — Canonical Verification Commands

```
## 13. Canonical Verification Commands

**Always use these exact commands.** Do NOT use `uv run pytest` (times out)
or invent variants.

### Backend Tests
```bash
# Run full test suite (canonical)
cd /path/to/project/backend
PYTHONPATH=. .venv/bin/pytest tests/ -q

# Run specific test file
PYTHONPATH=. .venv/bin/pytest tests/api/test_workflow_routes.py -v

# Run with output (for debugging)
PYTHONPATH=. .venv/bin/pytest tests/ -v -s

# Current baseline: 258 tests — never commit if count drops unexpectedly
```
```

Frontend Build + Type Check

```bash
# Full build (catches TypeScript errors)
cd /path/to/project/frontend
pnpm run build

# Type check only (faster, no output files)
pnpm run typecheck   # if available; otherwise use build
```

Alembic (Database Migrations)

```bash
# Check migration status (heads = current DB state)
cd /path/to/project/backend
.venv/bin/alembic heads
.venv/bin/alembic current

# Check for pending migrations
.venv/bin/alembic check

# Create new migration (autogenerate from models)
.venv/bin/alembic revision --autogenerate -m "description_here"

# Current migration chain head: 011 (next new migration = 012)
# Applied: 001 → 002+003 → merge(9754fd) → 004 → 005 → 006 → 007 → 008 →
009 → 010 → 011

# Apply migrations (CANNOT run from host — .env not present)
# Must apply via Docker:
docker exec -it blitz-postgres psql -U blitz blitz -c "<SQL from
```

```
migration>"
# Or: just migrate  (if .env is present on host)
```

LiteLLM / Docker Health

```
just ps                # all service status
just logs backend      # tail backend logs
curl http://localhost:8000/health  # backend health (no auth needed)
curl http://localhost:4000/health  # LiteLLM health
```

```
---

#### Enhancement E: New Section 14 — Critical Gotchas

```markdown
## 14. Critical Gotchas (Read Before Writing Code)

These are hard-won discoveries that apply to every session. They live here
so that
agents starting fresh don't have to rediscover them from STATE.md.

### Python / Backend

| Gotcha | Symptom | Fix |
|--------|---------|-----|
| `uv run pytest` times out | Hangs indefinitely | Use `.venv/bin/pytest`
directly |
| `uv run alembic` times out | Hangs indefinitely | Use `.venv/bin/alembic`
directly |
| Relative imports fail | `ImportError` in pytest | Use `PYTHONPATH=.
.venv/bin/pytest` |
| Alembic from host fails | `alembic.ini` can't find `.env` | Apply via
`docker exec psql` or `just migrate` |
| Two migrations branch from same revision | `alembic heads` shows multiple
heads | Create merge migration: `.venv/bin/alembic merge <rev1> <rev2>` |
| `FlagEmbedding` ImportError | `is_torch_fx_available removed` |
`transformers<5.0` pinned in pyproject.toml — do not upgrade |
| Celery tasks are sync | `async def` tasks silently don't await | Wrap:
`asyncio.run(_run())` inside each task |
| No FK constraint on `user_id` columns | Intended — users live in Keycloak
| Never add FK to `users` table (doesn't exist in PostgreSQL) |
| `JSON().with_variant(JSONB(), 'postgresql')` | Needed for SQLite test
compat | Use on all JSONB columns in ORM models |

### Frontend / TypeScript

| Gotcha | Symptom | Fix |
|--------|---------|-----|
```
```

```
| Next.js 15 async params | `params.id` type error | Type as `Promise<{id:
string}>`, await before use |
| FastAPI route ordering | String literals matched as UUID | Declare
`/templates`, `/runs/*` BEFORE `/{workflow_id}` |
| `pnpm run build` fails on `any` | TypeScript strict mode | Use `unknown`
+ type guard instead |
| CopilotKit agent name mismatch | Agent not found | Frontend must
reference exact name: `'blitz_master'` |
| `react-markdown` v10 removed `className` | TypeScript error | Wrap in
`<div className="...">` instead |

### Security / Auth

| Gotcha | Symptom | Fix |
|--------|---------|-----|
| Keycloak self-signed cert | JWKS fetch fails |
`KEYCLOAK_CA_CERT=frontend/certs/keycloak-ca.crt` in `backend/.env` |
| No `aud` claim in JWT | `JWTError` → 401 | `options={"verify_aud":
False}` in `jose_jwt.decode()` |
| Roles in `realm_roles` not `realm_access.roles` | Empty roles list → 403
| Check both: `payload.get("realm_roles") or payload.get("realm_access",
{}).get("roles", [])` |
| CopilotKit body is camelCase | `RunAgentInput(**body)` fails | Use
`RunAgentInput.model_validate(body)` |

### GSD-Specific

| Gotcha | Symptom | Fix |
|--------|---------|-----|
| Lazy imports not patchable | `patch('module.func')` doesn't work in tests
| Import at module top level; patch at definition module |
| `BlitzState` fields are `None` in LangGraph | Agent doesn't inject custom
state | Use contextvar fallback: `user_id = user_id_ctx.get(None)` |
| `classifyHandoffIfNeeded` error | Executor reports "failed" | Claude Code
runtime bug — spot-check SUMMARY.md + commits; if present, treat as success
|
```

Enhancement F: New Section 15 — Context Continuity Protocol

```
## 15. Context Continuity Protocol

How GSD and Superpowers maintain shared context across sessions.

### The /clear Checklist

Run this mental checklist before every `/clear`:
```

□ STATE.md updated with current position and any new decisions □ Any design docs (docs/plans/) committed to git □ If mid-task: /gsd:pause-work creates .planning/RESUME.md □ Technical discoveries added to docs/dev-context.md □ Known blockers added to STATE.md → Blockers section

```
### Session Start Protocol

```bash
# 1. Always start with:
/gsd:progress
# → Reads STATE.md + ROADMAP.md → shows position + routes to next action

# 2. If RESUME.md exists:
/gsd:resume-work
# → Restores exact stopping point from RESUME.md

# 3. Confirm context by reading what /gsd:progress reports
# Never jump to coding without knowing current position
```
```

Decision Recording Format

When making technical decisions during implementation, add to STATE.md:

```
## Accumulated Context

### Decisions
- [04-01]: owner_user_id on workflows is NULLABLE — template rows have owner_user_id=NULL
- [04-02]: compile_workflow_to_stategraph() returns uncompiled builder — caller injects checkpointer
- [04-03]: GraphInterrupt caught by type name ("Interrupt" in type(exc).__name__) — avoids fragile import
```

Format: `- [context-tag]: <decision statement> — <rationale>`

What Lives Where

Context type	Where to store	Read by
Technical decisions	`.planning/STATE.md` Decisions	All GSD agents
Blockers / concerns	`.planning/STATE.md` Blockers	All GSD agents, `/gsd:progress`
Pending todos	`.planning/STATE.md` Pending Todos	`/gsd:check-todos`
Design decisions (phase-specific)	`.planning/phases/NN-CONTEXT.md`	gsd-planner, gsd-plan-checker

Context type	Where to store	Read by
Technical research	`.planning/phases/NN-RESEARCH.md`	gsd-planner
Endpoint / URL gotchas	`docs/dev-context.md`	AGENT INSTRUCTIONS Step 1
Critical cross-cutting gotchas	`CLAUDE.md` Section 14	All agents (every session)
Design intent	`docs/plans/YYYY-MM-DD-design.md`	brainstorming, writing-plans
Implementation plan	`docs/plans/YYYY-MM-DD-plan.md`	executing-plans, subagent-driven

Handoff Anti-Patterns

Anti-pattern	Problem	Correct Approach
Jumping to code without `/gsd:progress`	Wrong phase, duplicate work	Always start with `/gsd:progress`
Decisions only in conversation	Lost on `/clear`	Write to STATE.md before clearing
Design docs not committed	Lost if session crashes	Commit before `/clear`
Gotchas only in SUMMARY.md	Future agents don't read SUMMARY	Add critical ones to CLAUDE.md §14
STATE.md updated by hand without commit	Uncommitted state lost	Commit STATE.md with each plan completion

```
---

### 6.5 Summary: What to Add vs. What to Enhance

| CLAUDE.md Location | Action | Priority |
|--------------------|--------|----------|
| AGENT INSTRUCTIONS | Add Steps 5–6 (decision recording + canonical
commands) | Critical |
| DO/DON'T tables | Add "Commits (GSD Tracking)" table | High |
| Section 12 | Replace with expanded version (mandatory rule + artifact
protocol + continuity) | High |
| New Section 13 | Add canonical verification commands | Critical |
| New Section 14 | Add critical gotchas quick reference | Critical |
| New Section 15 | Add context continuity protocol | High |
| Section 8 directory tree | Add `docs/plans/` entry | Low |

**Implementation order:** 13 → 14 → AGENT INSTRUCTIONS Steps 5-6 → DO/DON'T
commits table → Section 12 → 15

The total addition is approximately 200 lines added to CLAUDE.md. The
```

```
existing content (Sections 1–11) remains unchanged — these are additive
enhancements only.
```